

Array vs Linked List: Code chèn phần tử & giải thích

Tài liệu này cung cấp so sánh ngắn gọn giữa Array (Python list) và Linked List, kèm mã nguồn minh họa thao tác chèn phần tử và giải thích độ phức tạp.

Phần 1: Array (Python list)

Mã nguồn:

```
# Minh họa chèn phần tử vào Array (Python list)
arr = [10, 20, 30, 40]

# Chèn vào giữa (vị trí index = 2)
i = 2
x = 25
arr.insert(i, x) # arr -> [10, 20, 25, 30, 40]

# Chèn vào cuối
arr.append(50) # arr -> [10, 20, 25, 30, 40, 50]

print(arr)
# Ghi chú:
# - insert(i, x) dịch các phần tử từ i trở đi sang phải -> O(n)
# - append(x) thường O(1) amortized, đôi lúc O(n) nếu phải cấp phát lại vùng nhớ
```

Giải thích:

- Khởi tạo mảng ban đầu `arr = [10, 20, 30, 40]`.
- `arr.insert(2, 25)`: chèn giá trị 25 vào vị trí `index=2`, các phần tử bên phải dịch sang phải (độ phức tạp $O(n)$).
- `arr.append(50)`: chèn vào cuối mảng với chi phí trung bình $O(1)$ (amortized); đôi khi $O(n)$ nếu phải mở rộng capacity.
- `print(arr)`: kiểm tra mảng sau khi chèn: `[10, 20, 25, 30, 40, 50]`.

Phần 2: Linked List (Danh sách liên kết đơn)

Mã nguồn:

```
# Minh họa Linked List đơn và thao tác chèn
class Node:
    def __init__(self, value, next=None):
```

```

        self.value = value
        self.next = next

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_head(self, value):
        """Chèn đầu danh sách: O(1)"""
        new_node = Node(value, next=self.head)
        self.head = new_node

    def insert_after(self, node, value):
        """Chèn sau một node đã biết: O(1)"""
        if node is None:
            raise ValueError("Node tham chiếu không hợp lệ")
        new_node = Node(value, next=node.next)
        node.next = new_node

    def insert_at(self, index, value):
        """Chèn theo vị trí index: O(n) để duyệt đến index-1"""
        if index == 0:
            self.insert_head(value)
            return
        prev = self.head
        pos = 0
        while prev is not None and pos < index - 1:
            prev = prev.next
            pos += 1
        if prev is None:
            raise IndexError("Index vượt quá độ dài danh sách")
        self.insert_after(prev, value)

    def to_list(self):
        """Tiện ích chuyển sang Python list để xem kết quả"""
        out = []
        curr = self.head
        while curr:
            out.append(curr.value)
            curr = curr.next
        return out

# Sử dụng
ll = SinglyLinkedList()
ll.insert_head(30)          # Danh sách: 30
ll.insert_head(20)          # Danh sách: 20 -> 30
ll.insert_head(10)          # Danh sách: 10 -> 20 -> 30

```

```

# Chèn sau node có giá trị 20 (đã duyệt để lấy node tham chiếu)
p = ll.head          # 10
while p and p.value != 20:
    p = p.next        # p sẽ trở thành node có value 20
ll.insert_after(p, 25) # 10 -> 20 -> 25 -> 30

# Chèn theo index (ví dụ: index=2)
ll.insert_at(2, 22)   # 10 -> 20 -> 22 -> 25 -> 30

print(ll.to_list())   # [10, 20, 22, 25, 30]

```

Giải thích:

- Lớp Node lưu giá trị và con trỏ next, cho phép phân bổ rời rạc.
- `insert_head(value)`: gắn node mới làm head trong $O(1)$.
- `insert_after(node, value)`: chèn ngay sau node tham chiếu trong $O(1)$.
- `insert_at(index, value)`: cần duyệt đến index-1 ($O(n)$), sau đó chèn bằng `insert_after` ($O(1)$).
- `to_list()`: duyệt toàn bộ danh sách để chuyển sang Python list, hữu ích để kiểm tra kết quả.
- Ví dụ sử dụng: xây danh sách 10 -> 20 -> 30; chèn 25 sau node 20; chèn 22 tại index=2; kết quả [10, 20, 22, 25, 30].

Phần 3: Tóm tắt độ phức tạp

- Array: truy cập theo index $O(1)$; chèn giữa $O(n)$; chèn cuối $O(1)$ trung bình (amortized).
- Linked List đơn: chèn đầu/sau node $O(1)$; chèn theo index $O(n)$ (do phải duyệt); truy cập theo index $O(n)$.